

Support de présentation

Ce que nous disons sur chaque slide, et pourquoi nous avons fait ces choix

Projet	CryptoBot, bot de trading piloté par le machine learning
Formation	Data Engineer, DataScientest
Dépôt	DataScientest-Studio/jul26_bde_crypto
Date	1 octobre 2026

Sommaire

Déroulé

Slide 1 : CryptoBot

Slide 2 : De la donnée brute à une décision automatisée

Slide 3 : Architecture globale

Slide 4 : Collecter les données

Slide 5 : La bonne base pour la bonne forme

Slide 6 : Apprendre sans tricher

Slide 7 : Avoir raison ne suffit pas

Slide 8 : Du notebook au service protégé

Slide 9 : Ce qui fait tenir le service

Slide 10 : Démonstration en direct

Slide 11 : Tourner seul, remplacer avec preuve

Slide 12 : Du commit à la supervision

Slide 13 : Un système complet, une stratégie à améliorer

Slide 14 : Merci, questions

Pièges à éviter à l'oral

Déroulé

Deux présentateurs, deux blocs. A porte les données et le machine learning (étapes 1 à 3), B porte le déploiement et l'automatisation (étapes 4 et 5) avec la démonstration. Le bilan revient à B, qui vient de montrer le système en marche ; A conclut et ouvre les questions. Les durées sont indicatives : la démonstration est le seul bloc à tenir à la seconde.

Slide	Sujet	Durée	Présentateur
1	Titre	0 min 30	A
2	Le projet	1 min 00	A
3	Architecture globale	1 min 30	A
4	Étape 1 : collecter les données	1 min 15	A
5	Étape 2 : la bonne base pour la bonne forme	1 min 30	A
6	Étape 3 : apprendre sans tricher	2 min 00	A
7	Le résultat, sans l'enjoliver	1 min 30	A
8	Étape 4 : du notebook au service protégé	1 min 30	B
9	Étape 4 : ce qui fait tenir le service	1 min 15	B
10	Démonstration en direct	3 min 00	B
11	Étape 5 : tourner seul, remplacer avec preuve	1 min 30	B
12	Étape 5 : du commit à la supervision	1 min 30	B
13	Bilan	1 min 30	B
14	Merci, questions	0 min 30	A
Total		20 min 00	A : 9 min 45, B : 10 min 15

Repères de temps : A doit passer la main vers 9 min 45. Si la démonstration commence après 12 min 30, raccourcir les slides 11 et 12 à une minute chacune.

Slide 1 : CryptoBot

À dire

Bonjour. Nous allons vous présenter CryptoBot, notre projet de fin de cursus Data Engineer. C'est un bot de trading sur des cryptomonnaies, piloté par un modèle de machine learning. Nous l'avons construit en cinq étapes, de la collecte des données jusqu'à l'automatisation complète. Nous allons nous partager la parole : je présente la partie données et machine learning, puis mon binôme présente l'API, le déploiement, l'automatisation, et vous fera une démonstration en direct. Tout au long de la présentation, nous allons surtout vous expliquer pourquoi nous avons fait chaque choix, et ce que nous avons appris quand un choix s'est révélé faux.

Pourquoi ce choix

- Annoncer le découpage A puis B : le jury sait qui répondra à quoi (plutôt qu'alterner à chaque slide).
- Annoncer d'emblée qu'on parlera des erreurs : cela prépare la slide 7, qui montre un modèle non rentable.

Transition

Commençons par la question à laquelle le projet répond.

Slide 2 : De la donnée brute à une décision automatisée

À dire

Le point de départ est une question simple. Sur cinq paires de Binance, la prochaine bougie va-t-elle monter ou baisser ? Une bougie, c'est le résumé d'une tranche de temps : prix d'ouverture, plus haut, plus bas, clôture et volume. Le modèle donne une probabilité de hausse, et le bot en tire une décision : acheter, vendre ou attendre. Le bouton conservateur ou agressif règle à partir de quelle certitude il agit. Nous avons choisi de prédire un sens plutôt qu'un prix. Prédire le prix donne un score trompeur : répondre « le prochain prix sera le prix actuel » obtient un R^2 de 0,999998, et les vrais modèles font moins bien. Le droit d'attendre est aussi un choix : le modèle ne se prononce que quand il est assez sûr. Les cinq étapes que vous voyez suivent le parcours d'une donnée, de Binance jusqu'à la décision.

Pourquoi ce choix

- Prédire un sens : mesure honnête et lisible (prédire le prix, score faussement parfait).
- Pouvoir s'abstenir : mieux vaut 60 % de bonnes réponses sur peu de bougies que 52 % sur toutes (un modèle qui répond toujours).
- Cinq paires parmi les plus échangées : bougies régulières, sans trous dus à l'absence d'échanges (petites paires peu liquides).

Transition

Avant de détailler chaque étape, voici comment tout s'assemble.

Slide 3 : Architecture globale

À dire

Ce schéma se lit en trois bandes. En haut, la livraison du code : de GitHub jusqu'à la mise en service. Au milieu, l'application, dans Docker Compose : Binance, les deux bases, Airflow, MLflow, l'API et l'interface. En bas, la supervision. Ce découpage vient d'une idée simple : une

application de machine learning change pour trois raisons. Le code change, ce qui est le rôle de la bande du haut. Les données arrivent avec le temps, et le modèle vieillit quand le marché s'éloigne de ce qu'il a appris : c'est le rôle d'Airflow et de MLflow. La supervision observe les trois. Deux points à retenir sur ce schéma. L'API est au centre et n'est jamais exposée directement : tout passe par l'interface. Et tout tient sur une machine, en seize services lancés par une seule commande.

Pourquoi ce choix

- Trois bandes : une automatisation par raison de changer, code, données, modèle (un seul script qui ferait tout).
- Docker Compose : tout le projet démarre pareil sur n'importe quelle machine (installation à la main, « ça marche chez moi »).
- Profils Compose : le produit démarre toujours, pipeline, supervision et outils s'ajoutent à la demande (tout démarrer à chaque fois).

Transition

Remontons au début de la chaîne : d'où viennent les données.

Slide 4 : Collecter les données

À dire

Binance offre deux accès, qui répondent à deux questions. L'API REST dit ce qui s'est passé : c'est l'historique. Le WebSocket dit ce qui se passe maintenant : c'est le temps réel. Deux contraintes ont façonné le code. Binance renvoie au plus 1 000 bougies par appel, donc nous enchaînons les appels en repartant de la dernière bougie reçue, et non d'un compteur : si un lot est incomplet, un compteur sauterait des données sans le dire. Et nous lisons le quota consommé dans chaque réponse pour ne jamais être bannis. Côté WebSocket, Binance envoie la bougie en cours à chaque transaction : sur 200 secondes, 500 messages pour 20 bougies. Nous ne gardons que la version clôturée. Les deux flux arrivent dans un schéma commun, en UTC. Le résultat : un peu plus de deux millions de bougies, complètes à 100 %, sans doublon ni incohérence.

Pourquoi ce choix

- Pagination sur la dernière bougie reçue : juste même si Binance renvoie un lot incomplet (compteur d'intervalles).
- Garder seulement la bougie clôturée : sinon 96 % des lignes seraient des doublons à des prix jamais confirmés (tout enregistrer).
- Historique commun depuis septembre 2020 : SOLUSDT n'existe que depuis août 2020, on compare les paires sur la même période (BTC dès 2017).
- Ne pas combler les trous : le marché était fermé, inventer des prix créerait une information fausse (interpolation).

Transition

Ces bougies, il faut maintenant les ranger, et nous avons choisi deux bases.

Slide 5 : La bonne base pour la bonne forme

À dire

Notre premier réflexe a été de dimensionner. Deux millions de bougies, 132 mégaoctets : PostgreSQL ralentit vers quelques dizaines de millions de lignes, nous en sommes très loin. La performance brute n'était donc pas le critère. Nous avons choisi selon la forme de la donnée. Une bougie a toujours les mêmes treize colonnes et une clé unique : paire, pas de temps, heure d'ouverture. C'est du relationnel, dans PostgreSQL avec l'extension TimescaleDB, qui découpe les tables par tranches de temps et compresse l'ancien. Les réponses brutes de Binance et ses métadonnées ont des structures variables et imbriquées : elles vont dans MongoDB, telles qu'elles arrivent. Cette couche brute nous permet de rejouer un nettoyage corrigé sans redemander six ans de données. Chaque bougie garde une référence vers son document d'origine. Et le chargement est idempotent : le relancer ne crée jamais de doublon.

Pourquoi ce choix

- Répartir par forme de donnée : une bougie REST et une bougie WebSocket sont le même objet (« SQL pour l'historique, NoSQL pour le temps réel »).
- TimescaleDB : reste du PostgreSQL, compression et tranches réglées par profil (InfluxDB ou ClickHouse, faits pour des milliards de lignes et sans transactions ; Snowflake ou BigQuery, payants et non conteneurisables).
- Une table par profil, un pas de temps stocké une seule fois, une vue par profil : pas de doublon, et le code ignore où est rangée chaque bougie.
- Chargement idempotent (mise à jour sur la clé) : une collecte interrompue se relance sans risque, condition de l'automatisation.

Transition

Avec des données propres et traçables, nous pouvons entraîner un modèle, à condition de ne pas tricher avec le temps.

Slide 6 : Apprendre sans tricher

À dire

Sur des séries temporelles, l'erreur la plus fréquente est de laisser le futur entrer dans l'apprentissage. Nous l'avons faite. En ajoutant le contexte des bougies d'une heure et de quatre heures, notre score est passé de 0,52 à 0,61. Trop beau : une bougie de 12 h 15 recevait les indicateurs d'une bougie de quatre heures qui ne se terminait qu'à 16 h. Corrigé, le score est

retombé à 0,52, et un test automatique empêche ce retour. D'où les règles de cette slide. Le découpage suit le temps : passé pour apprendre, futur pour vérifier. Nos 35 variables sont sans unité, des rapports et des pourcentages, pour qu'un modèle appris sur le bitcoin reste valable sur l'ETH. Le modèle est une forêt aléatoire, calibrée pour que ses probabilités correspondent aux fréquences observées. Chaque style a deux seuils, un pour acheter et un pour vendre. Et la mesure finale porte sur 14 831 bougies que nous n'avons jamais regardées pendant les choix.

Pourquoi ce choix

- Découpage chronologique et validation croisée temporelle : on teste toujours sur la période suivante (découpage aléatoire, qui mélange février et mars).
- Variables sans unité, choisies à la main : 26 puis 35 variables (les 86 colonnes automatiques de la bibliothèque `ta`, dont 231 paires presque identiques).
- Forêt aléatoire calibrée : meilleur score en validation, la calibration apporte +1,5 à +1,7 point (gradient boosting, régression logistique).
- Deux seuils par style : la probabilité de hausse ne dépasse jamais 0,564, un seuil unique empêchait le style conservateur d'acheter (seuil symétrique autour de 0,5).

Transition

Voici maintenant ce que vaut ce modèle, sans l'enjoliver.

Slide 7 : Avoir raison ne suffit pas

À dire

Nous nous étions fixé un objectif : 60 % de bonnes réponses sur les bougies où le modèle se prononce. Il est atteint : 59,9 % en style conservateur, avec un intervalle de confiance de 55 à 64 %, donc nettement au-dessus du hasard. Mais le bot perd de l'argent : moins 3,6 % au backtest, frais compris. Sans frais, il aurait gagné environ 1 %. L'explication tient en deux chiffres. Une bougie de 15 minutes bouge en moyenne de 0,227 %. Un aller-retour sur Binance coûte 0,2 %. Le gain d'une bonne réponse est presque entièrement mangé par les frais : pour être rentable sur 15 minutes, il faudrait avoir raison 94 % du temps. Nous avons préféré montrer ce résultat tel quel plutôt que chercher la période qui arrange. Notre conclusion : le modèle sait un peu prédire le sens, mais il faudrait viser uniquement les mouvements assez grands pour couvrir les frais.

Pourquoi ce choix

- Accuracy sélective avec intervalle de confiance : sur 469 ordres, un chiffre seul ne dit rien de sa fiabilité (une accuracy brute sur toutes les bougies).
- Backtest frais compris (0,1 % à l'achat, 0,1 % à la vente) : seule mesure qui dit si la stratégie rapporte (backtest sans frais, environ +1 %).
- Rejeu sur un an : en 4 h, 53,4 % de trades gagnants et le capital finit à 10 154 euros ; en 15 min, 7 547 euros. Même taux de réussite, c'est le mouvement face aux frais qui change.

- Montrer l'échec : la soutenance juge la démarche, et un résultat trop beau aurait signalé une fuite.

Transition

Ce modèle, il faut maintenant le sortir du notebook. Je laisse la parole à mon binôme pour le déploiement.

Slide 8 : Du notebook au service protégé

À dire

Pour servir le modèle, nous avons écrit une API avec FastAPI. Le modèle y est chargé une fois en mémoire, et rechargé tout seul quand un nouveau fichier le remplace. La question principale a été la sécurité. Nous avons choisi une clé d'API par client : l'interface a la sienne, Airflow a la sienne, et l'une se révoque sans couper l'autre. Nous avons écarté JWT, fait pour des personnes qui se connectent avec un mot de passe ; nos clients sont des programmes. Problème : une clé envoyée au navigateur peut être lue par n'importe qui. Le navigateur parle donc à nginx, qui ajoute la clé et transmet à l'API. L'API, elle, n'a aucun port ouvert sur la machine. Elle est aussi fermée par défaut : si on oublie de configurer une clé, elle refuse tout au lieu de tout accepter. Enfin, paires, pas de temps et styles sont vérifiés par liste blanche, et nginx limite le débit.

Pourquoi ce choix

- Clé d'API par client : accès de programme à programme, comme Binance ou GitHub (JWT, pensé pour des utilisateurs humains).
- nginx en proxy qui ajoute la clé : la clé ne quitte jamais le serveur (clé dans le code JavaScript de la page).
- Fermée par défaut, réponse 503 sans clé configurée : un oubli ne peut pas ouvrir l'API (API ouverte tant qu'on n'a rien réglé).
- Limite de 10 requêtes par seconde par adresse, rafales de 40 : protège l'API et le quota Binance (pas de limite).

Transition

Une API protégée doit aussi tenir dans la durée : voici ce qui la rend fiable.

Slide 9 : Ce qui fait tenir le service

À dire

Trois choses font tenir le service. D'abord les tests : 149 tests unitaires, qui tournent en une dizaine de secondes. Ils remplacent les bases et le modèle par des doublures, pour qu'un échec désigne un bug du code et non une base éteinte. Un test parcourt toutes les routes et vérifie

qu'elles refusent un appel sans clé. Mais certaines choses n'existent qu'une fois les conteneurs assemblés, comme la clé ajoutée par nginx : 21 vérifications appellent donc l'application démarrée, comme un utilisateur. Ensuite les conteneurs : seize services, une commande, et le modèle monté à part pour être remplacé sans reconstruire l'image. Enfin la dérive. Nous la mesurons avec l'indice PSI, par paire et par pas de temps, car une référence mélangée annonçait une dérive alors que rien n'avait bougé. Résultat : tout le marché dérive, surtout la volatilité. Mais dérive des données ne veut pas dire baisse de performance, nous y reviendrons.

Pourquoi ce choix

- Doublures dans les tests : rapides, sans Docker, un échec pointe le code (tests branchés sur les vraies bases).
- 21 vérifications de l'application démarrée, dont une rafale de 120 appels où environ 75 sont refusés : couvre ce que les tests unitaires ne voient pas.
- PSI par paire et par pas de temps, fenêtre de 90 jours pour chacun : la taille moyenne d'un trade vaut 0,005 sur le bitcoin et 115,8 sur le XRP (référence globale, ou fenêtre en nombre de bougies).
- Modèle hors de l'image, en lecture seule : nouveau modèle sans reconstruction (modèle copié dans l'image).

Transition

Plutôt que de le décrire, voyons le service tourner.

Slide 10 : Démonstration en direct

À dire

Ce que vous allez voir est l'interface du bot, servie par nginx sur la machine. Elle affiche les bougies de Binance en direct, la décision du modèle sur la dernière bougie clôturée, et le carnet de positions virtuelles. Ce carnet existe parce que des pourcentages ne disent pas tout : quand le modèle donne un signal, le bot ouvre une position fictive avec un take profit et un stop loss placés à trois fois la volatilité récente, et une échéance de douze bougies. La position se ferme sur le premier des trois atteint. Si le take profit et le stop loss sont touchés dans la même bougie, nous retenons le stop loss, l'hypothèse la moins favorable. Rien n'est réellement acheté : c'est une simulation sur le marché spot.

Script de la démonstration (3 minutes)

Avant la soutenance : application démarrée depuis au moins une heure, onglets ouverts dans cet ordre : interface (localhost:8080), Grafana (localhost:3000), Airflow (localhost:8088), MLflow (localhost:5000). Interface réglée sur BTCUSDT, 1h, Conservateur, période 1 semaine, Trades coché, Données « Binance en direct ».

Temps	Ce qu'on clique	Ce qu'on dit
0:00	Onglet interface, rien à cliquer	« BTCUSDT en bougies d'une heure. La dernière bougie bouge en direct, elle vient du WebSocket de Binance. En haut : dernier prix, décision, probabilité de hausse, heure de la prochaine clôture. »
0:30	Montrer le panneau de décision et sa réglette	« Le modèle donne une probabilité de hausse. Ici, les deux seuils du style conservateur : au-dessus, il achète ; en dessous, il vend ; entre les deux, il attend. La plupart du temps, il attend. »
0:55	Bouton Agressif	« En agressif, les seuils se resserrent : 0,5346 pour acheter, 0,4315 pour vendre. Le modèle est le même, seule la certitude demandée change. »
1:20	Période : 1 mois	« Voici le carnet sur un mois. Chaque flèche est une ouverture de position. Les rectangles sont les zones de take profit et de stop loss. Chaque point est une fermeture, avec le gain ou la perte en pourcentage, frais déduits. »
1:55	Survoler deux ou trois points de fermeture	« Certains gagnent, d'autres perdent. Le bilan, sous le graphique, confirme ce que nous avons dit : à peu près une chance sur deux par trade, et les frais font pencher le total du mauvais côté. »
2:20	Paire : SOLUSDT, puis bouton 15m	« Même chose sur une autre paire et un autre pas de temps : le modèle est commun aux trois pas de temps du profil day trading. »
2:45	Revenir à 1h, conservateur	« Ces positions avancent toutes seules : Airflow appelle l'API toutes les 15 minutes pour les 30 combinaisons, même quand personne ne regarde. C'est ce que nous allons voir maintenant. »

Plan B si la démonstration plante

- Bougies Binance absentes (réseau coupé) : menu Données, choisir « Base du projet ». Le graphique se recharge depuis PostgreSQL ; dire « sans réseau, l'interface lit la base du projet ».
- Interface inaccessible : passer directement sur Grafana (localhost:3000), tableau « CryptoBot, production ». Montrer les décisions du bot, le taux de trades gagnants, le gain net moyen et le capital virtuel : on y voit le même carnet.
- Rien ne démarre : captures d'écran de l'interface et de Grafana, préparées la veille dans un dossier ouvert d'avance. Les commenter avec le même texte que le tableau ci-dessus, sans s'excuser plus d'une phrase.
- Dans tous les cas, ne pas déboguer en direct : au-delà de 20 secondes de blocage, passer au plan suivant.

Pourquoi ce choix

- Carnet de positions virtuelles : juger le modèle en euros et en trades, pas seulement en pourcentage de bonnes réponses (backtest seul, invisible).
- Stop loss retenu en cas d'égalité dans une bougie : les données ne disent pas lequel a été touché en premier, on prend l'hypothèse défavorable.
- Bibliothèque de graphique livrée avec le projet : la page marche sans Internet (chargement depuis un CDN).

Transition

Ce que vous venez de voir tourne sans nous : passons à l'automatisation.

Slide 11 : Tourner seul, remplacer avec preuve

À dire

L'étape 5 demande que l'application tourne sans que personne ne lance de commande. Airflow orchestre trois traitements : la collecte et le tour du bot toutes les 15 minutes, la dérive chaque matin, le réentraînement chaque dimanche. Airflow ne calcule rien lui-même : chaque tâche lance une commande du projet, celle qu'on lancerait à la main, testée ailleurs. La règle centrale est la suivante : un modèle réentraîné n'est pas forcément meilleur. Le remplacer à chaque fois pourrait dégrader la production sans que personne ne s'en aperçoive. Nous organisons donc un duel. Le modèle en service, le champion, et le nouveau, le challenger, sont mesurés sur les 21 mêmes derniers jours, que le challenger n'a jamais vus. Le nouveau ne passe que s'il fait au moins aussi bien. Deux duels ont eu lieu, et les deux fois l'ancien a gardé sa place : les écarts étaient dans la marge d'erreur, donc à valeur égale, on garde celui qui est en service.

Pourquoi ce choix

- Champion contre challenger : on ne remplace qu'avec une preuve, sur au moins 30 ordres (remplacement automatique par le plus récent).
- Extrait figé et signé SHA-256 : relancé trois fois, le duel donne les mêmes chiffres à la quatrième décimale (entraînement sur la base vivante).
- Publication par renommage du fichier : l'API ne lit jamais un fichier à moitié écrit, et l'ancien est archivé (écriture directe).
- Exécuteur local d'Airflow : quelques tâches toutes les 15 minutes (Celery, qui ajoute une file de messages et des machines inutiles ici).

Transition

Reste à faire évoluer le code lui-même, et à savoir quand quelque chose casse.

Slide 12 : Du commit à la supervision

À dire

Quand nous poussons du code sur la branche principale, GitHub Actions lance cinq contrôles en parallèle : analyse du code, les 149 tests, le chargement des DAG Airflow, la validité des configurations, et un démarrage complet de l'application sur une machine vierge. Le contrôle des DAG existe parce qu'un DAG qui ne se charge pas disparaît sans message d'erreur : la collecte s'arrêterait en silence. Si tout est vert, les images sont publiées, et Watchtower, sur notre machine, les récupère et remplace les conteneurs. C'est la machine qui va chercher la mise à jour

: rien n'est ouvert vers l'extérieur. Les bases ne sont jamais remplacées. Pour la supervision, Prometheus lit les métriques, et un petit exportateur traduit l'état du pipeline, qui est en base, en métriques. Douze alertes répondent à quatre questions : le service répond-il, les données arrivent-elles, le bot tourne-t-il, le modèle tient-il ? Je vous montre le tableau Grafana.

Pourquoi ce choix

- Watchtower, en mode pull : la machine va chercher les images (exécuteur GitHub installé sur la machine, risqué car le dépôt est public et du code de fork pourrait y tourner).
- Démarrage complet en CI avec des secrets neufs : prouve que le projet démarre ailleurs que chez nous.
- Exportateur séparé : les questions importantes (données fraîches, bot actif, dernier duel) sont en base, et il survit à une panne de l'API.
- Alertes avec durée minimale, de 2 minutes à 1 heure : un redémarrage de quelques secondes ne déclenche rien. Grafana lit la base avec un compte en lecture seule.

Partage d'écran Grafana (30 secondes)

Onglet Grafana, tableau « CryptoBot, production ». Montrer de haut en bas : l'état des services et les alertes en cours, le retard des données par série, l'évolution de la dérive, puis l'historique des réentraînements. Phrase à dire : « Tout ce que nous avons présenté se lit ici, au même endroit, et le tableau est décrit dans un fichier versionné : il est identique sur toute machine. »

Transition

Pour finir, ce que nous retenons de ce projet.

Slide 13 : Un système complet, une stratégie à améliorer

À dire

Ce qui fonctionne : la chaîne complète, de Binance jusqu'à la décision, tourne seule, l'API est protégée, et une modification du code arrive en production sans intervention. Ce que nous avons appris tient en trois idées. Valider dans le temps : nos trois plus beaux résultats étaient faux, dont une fuite de données. Mesurer après les coûts : un modèle qui a raison six fois sur dix peut perdre de l'argent. Superviser la donnée en plus du serveur : nos alertes les plus utiles disent si les données arrivent et si le bot tourne. Pour la suite, trois pistes. Changer de cible pour ignorer les petits mouvements, que les frais rendent inexploitable. Ajouter des notifications, car aujourd'hui les alertes se consultent mais personne n'est prévenu. Et sortir d'une seule machine, qui reste le point faible : si elle s'arrête, tout s'arrête.

Pourquoi ce choix

- Changer de cible plutôt que de données : les pistes testées (calendrier, carnet d'ordres, horizons plus longs) n'ont rien apporté de régulier.

- Notifications avec Alertmanager vers un courriel ou une messagerie : complément naturel des 12 alertes existantes.
- Hébergement sur un serveur avec sauvegarde des volumes, et HTTPS devant nginx avant toute ouverture au réseau.

Transition

Je rends la parole à mon binôme pour conclure.

Slide 14 : Merci, questions

À dire

Merci de nous avoir écoutés. En une phrase : nous avons construit une chaîne de données et de machine learning complète, automatisée et supervisée, et nous l'avons mesurée honnêtement, jusqu'à montrer que la stratégie n'est pas encore rentable. Les quatre mots en bas de la slide résument les blocs auxquels nous pouvons répondre : données, machine learning, API, MLOps. Pour les questions sur la collecte, les bases et le modèle, je répondrai ; pour l'API, le déploiement et l'automatisation, ce sera mon binôme. Si vous voulez revoir une partie du système, l'application tourne toujours sur cette machine et nous pouvons rouvrir l'interface, Grafana, Airflow ou MLflow. Nous sommes à votre écoute.

Pourquoi ce choix

- Rappeler qui répond à quoi : évite de se couper la parole pendant les questions.
- Garder les onglets ouverts : une question sur le registre des modèles se montre dans MLflow, une question sur un DAG dans Airflow.

Transition

Laisser la première question venir, sans relancer.

Pièges à éviter à l'oral

- Ne jamais dire que le bot est rentable. Dire « 60 % de bonnes réponses quand il se prononce, mais moins 3,6 % au backtest frais compris ».
- Ne pas confondre dérive des données et baisse de performance. Toutes les séries dérivent, et le modèle en service gardait 57,5 % de bonnes réponses, dans la marge des 60 %. La dérive déclenche une comparaison, jamais une mise en service automatique.
- Dire « marché spot », et préciser que le projet est académique : aucun ordre réel n'est passé, les positions sont virtuelles. Ne pas parler de contrats à terme ni d'effet de levier.
- Ne pas présenter 60 % comme une certitude : c'est une moyenne, avec un intervalle de confiance de 55 à 64 %.

- Ne pas dire que l'API est « sécurisée » sans nuance : elle n'a pas de HTTPS, ce qui est acceptable tant que tout reste sur la machine locale, et les secrets sont dans un fichier `.env`.
- Ne pas dire qu'Airflow entraîne le modèle : il lance les commandes du projet, la logique est dans les scripts testés.
- Ne pas lire les slides. Chaque chiffre affiché doit venir avec sa raison : pourquoi ce choix, et ce que nous avons écarté.
- Pendant la démonstration, ne pas déboguer en direct : passer au plan B au bout de 20 secondes.